

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 3

Compte officiel et diffusion

Messages et statuts adressés à une partie de la base

Ciblage composable, livraison paresseuse, file de jobs



Périmètre	Alanya-Backend ▪ Alanya Admin ▪ Talky (Flutter)
Statut	Conception — aucune ligne de code écrite
Dépend de	Socle de compte (migration 023)
Remplace	talky-admin/ideas/broadcast.tex
Échelle visée	Centaines de milliers de comptes
Date	26 juillet 2026

Avant-propos

Ce document remplace `talky-admin/ideas/broadcast.tex`, qui décrivait la même fonctionnalité avec une stratégie de livraison incompatible avec l'échelle visée. Les deux documents divergeaient aussi du §9 de `alanya-contexte-conception.md`; l'arbitrage est tranché ici, chapitre [4](#).

Comment lire ce document. La **partie I** décrit la fonctionnalité telle que l'administrateur et l'utilisateur la voient. La **partie II** décrit comment on la construit. On peut valider la partie I sans lire la partie II.

Table des matières

Avant-propos	1
I Approche naïve	4
1 Ce que l'on ajoute	5
1.1 Le besoin	5
1.2 Ce qui a déjà été décidé	5
2 Cibler une diffusion	6
2.1 Un critère, pas un choix parmi trois	6
2.2 Savoir qui recevra, avant d'envoyer	6
2.3 L'historique	7
3 Ce que voit l'utilisateur	8
3.1 Un message diffusé	8
3.2 Un statut diffusé	8
II Approche technique	9
4 La stratégie de livraison	10
4.1 Le désaccord à trancher	10
4.2 Les trois options, chiffrées	10
4.3 Le moment de la matérialisation	11
4.4 Le garde-fou contre la dérive	11
5 Modèle de données	12
5.1 Migration 025_broadcast.sql	12
6 Le résolveur de critères	14
6.1 Format stocké	14
6.2 Deux évaluateurs pour un même critère	14
6.3 Sécurité : compiler n'est pas concaténer	15
6.4 Ce qu'on n'accepte pas comme critère	15
7 Les statuts officiels	16
7.1 Le correctif de visibilité	16

7.2	Les statuts n'ont pas besoin de matérialisation	16
8	La file de jobs	17
8.1	Pourquoi elle devient obligatoire ici	17
8.2	Table plutôt que Redis	17
8.3	Consommation concurrente	17
8.4	Découpage du fan-out	18
9	Impact fichier par fichier	19
9.1	Backend	19
9.2	Administration	19
9.3	Mobile	20
10	Points de vigilance et questions ouvertes	21
10.1	Chantier d'infrastructure nommé	21
10.2	Coût de la matérialisation sur le chemin chaud	21
10.3	Questions ouvertes	21

PARTIE I

Approche naïve

CHAPITRE 1

Ce que l'on ajoute

1.1 Le besoin

Alanya n'a aujourd'hui aucun moyen de s'adresser à ses utilisateurs. Annoncer une maintenance, une nouvelle version ou prévenir un commerçant que sa vérification expire suppose de passer par un canal extérieur — e-mail, réseaux sociaux — que la plupart des utilisateurs ne consultent pas.

On ajoute donc un **compte officiel** capable d'envoyer deux choses :

- des **messages**, qui apparaissent comme une conversation ordinaire ;
- des **statuts**, qui apparaissent dans le fil des statuts.

Dans les deux cas, à **tous** les utilisateurs ou à une **partie** d'entre eux.

1.2 Ce qui a déjà été décidé

Le socle de compte (étape 1) a posé les règles qui encadrent ce compte :

Création	Par un administrateur, jamais par une inscription ordinaire.
Apparence	Avatar = logo Alanya, badge officiel doré, distinct du badge de vérification.
Réponse	Impossible. Le compte diffuse, il ne converse pas.
Nombre	Le modèle en supporte plusieurs (support, actualités...) sans modification.

CHAPITRE 2

Cibler une diffusion

2.1 Un critère, pas un choix parmi trois

La spécification d'origine prévoyait trois cibles : tous, par pays, ou une liste de comptes choisis. C'est insuffisant — on veut aussi pouvoir s'adresser aux commerçants, aux comptes inscrits depuis plus de trois mois, à ceux vus récemment, et à d'autres critères qu'on ne connaît pas encore.

Le ciblage devient donc une **combinaison de conditions**, que l'administrateur assemble. Ajouter un critère demain ne demandera aucune modification de la base.

Pays	Un ou plusieurs pays.
Genre de compte	Personnel, business, officiel.
État de vérification	Vérifié, en attente, expiré. . .
Ancienneté	Inscrit avant ou après une date, ou depuis une durée.
Activité	Vu dans les N derniers jours.
Comptes choisis	Une liste explicite, pour un message de service.

2.2 Savoir qui recevra, avant d'envoyer

À chaque modification du ciblage, l'administrateur voit l'estimation se recalculer. C'est le garde-fou principal : on ne diffuse pas à trois cent mille personnes sans l'avoir vu écrit.

Nouvelle diffusion

Le contenu à gauche, le ciblage à droite. L'estimation se recalcule à chaque changement.

NATURE

Message

Statut (24 h)

CONTENU

Bonjour ! La version 2.4 arrive la semaine prochaine. Les commerçants vérifiés pourront enfin publier leur catalogue directement depuis leur fiche. ...

APERÇU CÔTÉ UTILISATEUR

Le message apparaît dans une conversation avec le compte Alanya, sans zone de saisie. Voir la maquette du socle de compte.

Diffuser maintenant

CIBLAGE

☒

Pays est Cameroun

users.idPays = 5

41 %

ET

☒

Genre de compte est business

users.account_type = 1

4 %

ET

☐

Inscrit depuis plus de 3 mois

users.created_at

62 %

ET

☐

Vu dans les 30 derniers jours

users.last_seen

47 %

Ajouter une condition

DESTINATAIRES ESTIMÉS

4 880

sur 312 480 comptes non bannis

⚠ Diffusion large : le fan-out des notifications sera étalé par lots par la file de jobs. La conversation, elle, n'est créée qu'à la première ouverture de chaque destinataire.

Maquette interactive. Les conditions s'activent et se désactivent, l'estimation et la requête produite suivent :

claude.ai/code/artifact/b609a75d

Source versionnée : [docs/architecture/maquettes/diffusion-admin.html](#)

2.3 L'historique

Chaque diffusion conserve son critère, et non sa liste de destinataires. Six mois plus tard, on peut relire à qui elle s'adressait — ce qu'une liste d'identifiants ne dirait pas.

L'historique affiche aussi le taux d'ouverture, dérivé du nombre de destinataires ayant réellement rouvert l'application depuis l'envoi.

La relance J-30 passe par ce canal. Les avertissements d'expiration de vérification conçus à l'étape 1 ne sont rien d'autre qu'une diffusion dont le critère est « business, vérifié, échéance dans moins de 30 jours ». Aucun mécanisme spécifique n'est construit pour eux.

CHAPITRE 3

Ce que voit l'utilisateur

3.1 Un message diffusé

Une conversation ordinaire avec le compte Alanya, à ceci près que la zone de saisie est remplacée par un bandeau : « Ce compte diffuse des annonces. Vous ne pouvez pas y répondre. » Les actions « répondre » et « réagir » n'apparaissent pas sur ses messages.

Tout le reste fonctionne normalement : notification, compteur de non-lus, conservation hors ligne, recherche. C'est voulu — voir chapitre 4.

3.2 Un statut diffusé

Il apparaît dans le fil des statuts, avec l'avatar Alanya et le badge officiel, exactement comme le statut d'un contact. Il expire au bout de 24 heures comme les autres.

Un obstacle que la spécification d'origine n'avait pas vu. Aujourd'hui, un statut n'est visible que si l'auteur et le lecteur sont **mutuellement** en contacts préférés. En l'état, un statut officiel serait invisible pour absolument tout le monde — le compte Alanya ne peut pas ajouter trois cent mille personnes en favoris, ni espérer qu'elles le fassent en retour. Le correctif tient en trois lignes et se trouve au chapitre 7.

PARTIE II

Approche technique

CHAPITRE 4

La stratégie de livraison

4.1 Le désaccord à trancher

Deux documents existants se contredisaient :

<code>ideas/broadcast.tex</code>	Pour chaque destinataire : créer ou réutiliser la conversation 1-1, insérer une ligne <code>message</code> , incrémenter <code>unreadCount</code> , envoyer le push. C'est un <i>fan-out on write</i> .
<code>alanya-contexte-conception</code> §9	« Fan-out on read, pas on write. Un enregistrement de diffusion unique + un état de lecture par utilisateur, matérialisé à la demande . »

En lisant les deux attentivement, la contradiction est moins frontale qu'elle en a l'air : le §9 ne dit pas de renoncer aux lignes de message, il dit de ne pas les écrire *au moment de l'envoi*. C'est une question de *quand*, pas de *quoi*.

4.2 Les trois options, chiffrées

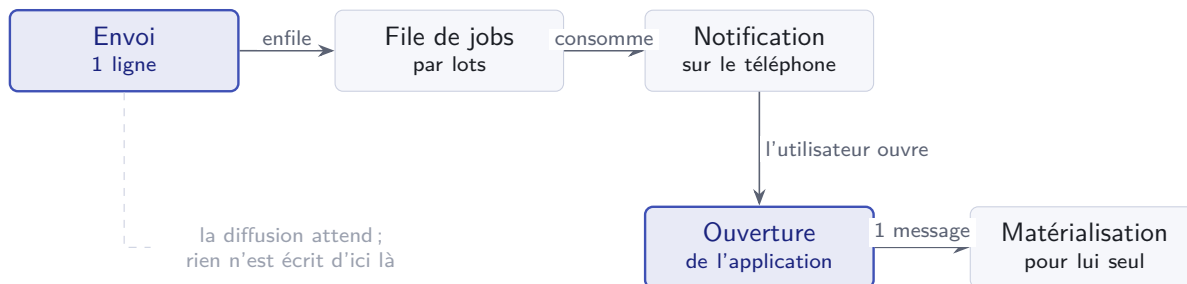
	Écriture immédiate	Matérialisation paresseuse	Lecture pure
À l'envoi	N conversations + N messages	1 ligne	1 ligne
Pipeline existant	réutilisé	réutilisé	à réécrire
Cache Drift	gratuit	gratuit	à refaire
Non-lus, accusés	gratuits	gratuits	à refaire
Effort	faible	moyen	élevé
Tient à 300 k	non	oui	oui

Décision : matérialisation paresseuse. À 300 000 comptes, l'écriture immédiate produirait 300 000 lignes `message` en une transaction. La lecture pure, elle, obligerait à réécrire tout le chemin de livraison — synchronisation, cache local, compteurs de non-lus, accusés de réception — pour un seul type de message.

4.3 Le moment de la matérialisation

Vérifié dans le code. `getMessagesSince` (`src/controllers/messageController.js`, ligne 922) ne synchronise que les conversations dont **le client envoie déjà le curseur**, et sort immédiatement en `{messages: [], hasMore: false}` si la liste est vide (ligne 941). Une conversation créée côté serveur n'est donc jamais découverte par ce chemin.

Le point d'accroche est donc `GET /api/conversations` — la liste des conversations, qui est le seul endroit où une conversation inconnue du client peut apparaître. Avant de répondre, on matérialise les diffusions en attente pour cet utilisateur.



4.4 Le garde-fou contre la dérive

Un critère évalué à la lecture pose un problème que l'écriture immédiate n'avait pas : un compte **créé après** la diffusion correspondrait au critère et recevrait un message qui ne lui était pas destiné.

Une clause suffit :

```
AND u.created_at <= b.sent_at
```

Pourquoi ne pas figer l'audience. On pourrait enregistrer la liste des destinataires à l'envoi. Ce serait exactement le *fan-out on write* qu'on vient d'écarter, avec les mêmes 300 000 lignes. La clause ci-dessus couvre le seul cas réaliste de dérive. Les autres — un compte devenu business après l'envoi, par exemple — sont acceptés : le critère décrit à *qui le message s'adresse*, et cette description reste vraie.

CHAPITRE 5

Modèle de données

5.1 Migration 025_broadcast.sql

```
1  -- 025_broadcast.sql -- compte officiel, diffusions et file de jobs
2  -- Requiert 023 (socle de compte) et 024 (listes de contacts).
3
4  CREATE TABLE IF NOT EXISTS broadcast (
5      id          BIGINT      NOT NULL AUTO_INCREMENT,
6      sender_id   INT         NOT NULL,      -- compte officiel
7              (account_type = 2)
8      created_by  INT         NOT NULL,      -- admin auteur de la
9              diffusion
10     kind         TINYINT     NOT NULL DEFAULT 0 COMMENT '0=message
11              1=statut',
12     content      TEXT        NULL,
13     type         TINYINT     NOT NULL DEFAULT 0 COMMENT '0=texte
14              1=image 2=video',
15     media_url    VARCHAR(255) NULL,
16     criteria     JSON        NOT NULL      COMMENT 'Critere de ciblage,
17              compile a la volee',
18     estimate     INT         NOT NULL DEFAULT 0 COMMENT 'Destinataires
19              estimees a l envoi',
20     statut_id    INT         NULL          COMMENT 'Ligne statut creee
21              (kind = 1)',
22     sent_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
23     PRIMARY KEY (id),
24     KEY idx_broadcast_sent (sent_at),
25     CONSTRAINT fk_bc_sender FOREIGN KEY (sender_id) REFERENCES
26         users(alanyaID),
27     CONSTRAINT fk_bc_admin  FOREIGN KEY (created_by) REFERENCES
28         users(alanyaID),
29     CONSTRAINT fk_bc_statut FOREIGN KEY (statut_id) REFERENCES
30         statut(ID) ON DELETE SET NULL
31 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
32
33 -- Une ligne par destinataire AYANT REELLEMENT reçu : creee a la
34 -- materialisation, pas a l'envoi. Donne le taux d'ouverture
35      gratuitement.
36
37 CREATE TABLE IF NOT EXISTS broadcast_delivery (
38     broadcast_id BIGINT NOT NULL,
39     alanyaID     INT    NOT NULL,
40     conversation_id BIGINT NULL,
41     message_id   BIGINT NULL,
42     delivered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
43     PRIMARY KEY (broadcast_id, alanyaID),
44     KEY idx_bd_user (alanyaID),
45     CONSTRAINT fk_bd_broadcast FOREIGN KEY (broadcast_id)
```

```
34 REFERENCES broadcast(id) ON DELETE CASCADE ,
35 CONSTRAINT fk_bd_user FOREIGN KEY (alanyaID)
36 REFERENCES users(alanyaID) ON DELETE CASCADE
37 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
38
39 -- Filigrane : jusqu'ou cet utilisateur a deja ete servi.
40 ALTER TABLE users
41 ADD COLUMN last_broadcast_id BIGINT NOT NULL DEFAULT 0
42 COMMENT 'Derniere diffusion evaluee pour ce compte (materialisee
    ou non)';
```

Listing 5.1 – Diffusions et livraisons

Pourquoi un filigrane sur users. Sans lui, chaque GET /api/conversations devrait comparer l'utilisateur à toutes les diffusions jamais envoyées. Avec lui, la requête ne remonte que `broadcast.id > u.last_broadcast_id`. Le filigrane avance **même quand le critère ne correspond pas** — on ne réévalue jamais deux fois la même diffusion pour le même compte.

Le taux d'ouverture sort tout seul. `broadcast_delivery` n'existe que pour les comptes ayant rouvert l'application depuis l'envoi. Le taux affiché dans l'historique est donc `COUNT(delivery) / broadcast.estimate` — aucune instrumentation supplémentaire.

CHAPITRE 6

Le résolveur de critères

6.1 Format stocké

```
1 {
2   "v": 1,
3   "op": "and",
4   "conditions": [
5     { "field": "pays",      "op": "in",      "value": [5, 7] },
6     { "field": "account_type", "op": "eq",      "value": 1 },
7     { "field": "created_at",  "op": "before", "value": "-3 months" },
8     { "field": "last_seen",   "op": "after",  "value": "-30 days" }
9   ]
10 }
```

Listing 6.1 – Exemple de valeur de `broadcast.criteria`

Le champ `v` porte la version du format : une diffusion relue dans deux ans doit rester interprétable même si le vocabulaire a évolué.

La règle du §7 sur le JSON est respectée, pas contournée. `alanya-contexte-conception.md` déconseille les colonnes JSON « dès qu'elles font l'objet d'un `WHERE` ». Ici, `criteria` n'est **jamais** filtré en SQL : il est lu, puis *compilé* en clauses. Aucune requête ne cherche à l'intérieur.

6.2 Deux évaluateurs pour un même critère

C'est le point qui rend l'ensemble simple. Le résolveur a deux modes, et l'un des deux n'écrit pas de SQL du tout :

Mode	Usage	Fonctionnement
Compilation SQL	estimation, parcours pour le push	Produit un fragment <code>WHERE</code> paramétré, appliqué sur <code>users</code> .
Évaluation en mémoire	« ce compte précis correspond-il ? »	Pure fonction JavaScript sur la ligne <code>users</code> déjà chargée. Aucune requête.

La matérialisation et la visibilité des statuts n'ont besoin que du **second** mode : l'utilisateur est connu, sa ligne est déjà en main, il suffit de tester ses attributs.

Invariant à tester. Les deux modes doivent toujours répondre la même chose. C'est exactement le genre d'accord qui se rompt silencieusement à la première évolution. Un test qui, pour un jeu de comptes, compare le résultat des deux évaluateurs sur chaque critère du catalogue est indispensable.

6.3 Sécurité : compiler n'est pas concaténer

Un critère venu du client, compilé en SQL, est une surface d'injection s'il est traité naïvement.

- Le champ (`field`) est résolu par une **liste blanche** vers un nom de colonne. Toute valeur inconnue est un rejet, pas un passage.
- L'opérateur vient d'une énumération fermée par type de champ.
- La valeur est **toujours** un paramètre lié (?), jamais interpolée.

Ce que le résolveur produit

Le critère est stocké en JSON puis **compilé** en clauses SQL. Il n'est jamais filtré en base — la règle du §7 sur les colonnes JSON est respectée.

```
-- critere stocke sur la ligne broadcast, compile a la volee
SELECT COUNT(*) FROM users u
WHERE u.exclus = 0
  AND u.created_at <= b.sent_at -- garde anti-derive
  AND u.idPays = 5
  AND u.account_type = 1;
```

Ce comptage n'est demandé qu'ici. À l'envoi, la liste des destinataires n'est jamais construite : le critère ne sert qu'à ce `COUNT(*)` et au fan-out push par lots.

Historique

Chaque diffusion conserve son critère, pour qu'on puisse relire six mois plus tard à qui elle s'adressait.

ENVOYÉE	NATURE	CONTENU	CIBLE	DESTINATAIRES	OUVERTURES
24 juil. 09:12	• Message	Maintenance programmée dimanche de 2 h à 4 h	pays = Cameroun	128 430	71 %
19 juil. 17:40	• Statut	Les listes de contacts arrivent	tous	309 004	44 %
11 juil. 08:05	• Message	Votre vérification expire dans 30 jours	business A vérifié A échéance < 30 j	1 246	92 %

6.4 Ce qu'on n'accepte pas comme critère

Tous les champs retenus sont indexés :

Critère	Colonne	Index
Ancienneté	<code>users.created_at</code>	<code>idx_users_created_at</code> (migration 005)
Activité	<code>users.last_seen</code>	<code>idx_users_online</code> (migration 001)
Pays	<code>users.idPays</code>	clé étrangère
Genre, état	<code>account_type</code> , <code>verification_status</code>	<code>idx_users_account_type</code> (migration 023)

Critère écarté. « Nombre de messages envoyés » exigerait une agrégation sur `message`, la plus grosse table de la base, à chaque estimation. À écarter, ou à pré-calculer dans une colonne entretenue — pas à compiler à la volée.

CHAPITRE 7

Les statuts officiels

7.1 Le correctif de visibilité

État actuel. `getStatus` (`src/controllers/statutController.js`, lignes 29–30) impose deux jointures INNER sur `preferredContact` : l’auteur doit avoir le lecteur en favori **et** réciproquement.

`ideas/broadcast.tex` proposait d’ajouter une colonne `isBroadcast` sur `statut`. Elle est inutile depuis le socle : l’information est déjà sur l’auteur.

```
1 FROM statut s
2 JOIN users u ON s.alanyaID = u.alanyaID
3 LEFT JOIN preferredContact pc1 ON pc1.alanyaID = s.alanyaID AND
   pc1.idFriend = ?
4 LEFT JOIN preferredContact pc2 ON pc2.alanyaID = ? AND
   pc2.idFriend = s.alanyaID
5 WHERE s.expiredAt > NOW()
6 AND (
7     u.account_type = 2 -- officiel : pas de
   reciprocite exigee
8 OR (pc1.idPrefContact IS NOT NULL -- sinon : regle inchangee
9 AND pc2.idPrefContact IS NOT NULL)
10 )
11 ...
```

Listing 7.1 – `getStatus` — jointures assouplies

7.2 Les statuts n’ont pas besoin de matérialisation

Asymétrie qu’il faut voir pour ne pas sur-concevoir : un statut est **une seule ligne lue par tout le monde**. Il est nativement en *fan-out on read*. Un message, lui, vit dans une conversation propre à chaque destinataire.

Un statut officiel se réduit donc à : une ligne `statut` portée par le compte officiel, plus le filtre de critère appliqué au lecteur — en mode évaluation en mémoire, sur la ligne `users` déjà chargée par la requête. Aucune table, aucun travail de fond.

CHAPITRE 8

La file de jobs

8.1 Pourquoi elle devient obligatoire ici

Quelle que soit la stratégie de livraison, envoyer les **notifications** reste un parcours de N destinataires. C'est le seul véritable fan-out qui subsiste, et à 300 000 comptes il ne peut pas tenir dans le cycle d'une requête HTTP.

Nuance à ne pas gommer. On lit souvent que la matérialisation paresseuse évite de parcourir les destinataires. C'est faux : elle évite de leur **écrire des lignes de message**. Le parcours pour le push existe bien — mais il se fait par pages, en arrière-plan, sans rien persister.

8.2 Table plutôt que Redis

Le §7 de `alanya-contexte-conception.md` recommandait BullMQ sur Redis. On retient une **table MySQL** : aucun service supplémentaire à héberger, sauvegarder et surveiller, pour un besoin qui se limite aujourd'hui au push par lots.

```
1 CREATE TABLE IF NOT EXISTS job_queue (  
2   id          BIGINT      NOT NULL AUTO_INCREMENT,  
3   kind        VARCHAR(40) NOT NULL COMMENT 'broadcast_push, ...',  
4   payload     JSON        NOT NULL,  
5   run_after   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6   attempts    TINYINT     NOT NULL DEFAULT 0,  
7   max_attempts TINYINT     NOT NULL DEFAULT 5,  
8   locked_at   DATETIME    NULL,  
9   locked_by   VARCHAR(64) NULL,  
10  failed_at   DATETIME    NULL,  
11  last_error  TEXT         NULL,  
12  created_at  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13  PRIMARY KEY (id),  
14  KEY idx_job_pret (kind, failed_at, locked_at, run_after)  
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Listing 8.1 – File de jobs — même migration 025

8.3 Consommation concurrente

SKIP LOCKED (MySQL 8, déjà la cible du schéma) permet à plusieurs consommateurs de prendre des lots disjoints sans se bloquer :

```
START TRANSACTION;  
SELECT id, payload FROM job_queue  
WHERE kind = ? AND locked_at IS NULL AND failed_at IS NULL
```

```
    AND run_after <= NOW()
  ORDER BY run_after, id
  LIMIT 20
  FOR UPDATE SKIP LOCKED;

UPDATE job_queue
SET locked_at = NOW(), locked_by = ?, attempts = attempts + 1
WHERE id IN (...);
COMMIT;
```

Un job réussi est supprimé ; un job échoué est déverrouillé avec `run_after` repoussé (recul exponentiel) jusqu'à `max_attempts`, puis marqué `failed_at` et laissé pour inspection.

8.4 Découpage du fan-out

L'enfileur parcourt les destinataires par **pagination par clé** sur `alanyaID` — jamais par `OFFSET`, qui se dégrade sur des centaines de milliers de lignes — et crée un job par lot de quelques centaines. Une diffusion à 300 000 comptes produit quelques centaines de jobs, pas 300 000.

Réutiliser l'existant. `src/services/meetingScheduler.js` fournit déjà le squelette d'une boucle de fond (`setInterval` d'une minute, démarrage depuis `server.js`). Le consommateur de jobs suit le même patron. `notificationService.js` envoie déjà les push par lots.

CHAPITRE 9

Impact fichier par fichier

9.1 Backend

Fichier	Nature	Objet
migrations/025_broadcast.sql	nouveau	broadcast, broadcast_delivery, job_queue, filigrane
src/services/criteriaResolver.js	nouveau	Compilation SQL <i>et</i> évaluation en mémoire
src/services/jobQueue.js	nouveau	Enfilage, consommation SKIP LOCKED, recul exponentiel
src/services/broadcastService.js	nouveau	Envoi, enfilage du push, matérialisation
src/controllers/admin/broadcast.js	nouveau	CRUD, estimation, historique
src/routes/admin.js	modifié	GET / POST /broadcasts, GET /broadcasts/:id, POST /broadcasts/estimate
src/controllers/conversationController.js	modifié	Matérialisation avant de répondre à la liste
src/controllers/statutController.js	modifié	Jointures en LEFT + critère
server.js	modifié	Démarrage du consommateur de jobs

9.2 Administration

L'écran existe déjà, branché sur des données simulées (app/(dashboard)/broadcasts/page.tsx, components/broadcasts/*, hooks/useBroadcasts.ts → lib/mock-data.ts). Il faut le brancher et remplacer le ciblage :

Fichier	Objet
components/broadcasts/BroadcastDialogForm	Remplacer les trois boutons radio targetType par le constructeur de conditions
components/broadcasts/CriteriaBuilderAjust (nouveau)	Ajust, retrait et édition des conditions
components/broadcasts/RecipientEstimateForm (nouveau)	Estimation en direct, appel anti-rebond sur /broadcasts/estimate
types/index.ts	targetType / targetCriteria deviennent criteria
lib/mock-data.ts	Retirer la bascule simulée pour les diffusions

Déjà anticipé. `BroadcastFormData` contient déjà un booléen `isStatus` (`types/index.ts`, ligne 324) : la distinction message / statut avait été prévue.

9.3 Mobile

Peu de travail, et c'est la récompense du choix de livraison : une diffusion arrive comme un message ordinaire.

Fichier	Objet
<code>lib/screens/chats/chat/chat_input.dart</code>	Bandeau à la place de la saisie (étape 1)
<code>lib/screens/chats/chat/chat_bubbles.dart</code>	Retrait de « répondre » et « réagir » (étape 1)
<code>lib/screens/status/statuses_screen.dart</code>	Aucun changement — le statut officiel arrive par la requête corrigée

CHAPITRE 10

Points de vigilance et questions ouvertes

10.1 Chantier d'infrastructure nommé

La **file de jobs** n'existe pas aujourd'hui dans le backend. Ce n'est pas un détail d'implémentation : c'est une brique à construire, tester et surveiller, avec sa propre supervision (jobs en échec, jobs bloqués par un consommateur mort). À budgéter comme telle.

Verrous orphelins. Un consommateur tué laisse ses jobs `locked_at` non nul indéfiniment. Il faut une reprise : tout job verrouillé depuis plus de N minutes est considéré comme abandonné et redevient disponible.

10.2 Coût de la matérialisation sur le chemin chaud

`GET /api/conversations` est appelé à chaque ouverture de l'application. Y ajouter un travail conditionnel demande de la rigueur : la requête de filigrane doit être triviale quand il n'y a rien à faire — c'est-à-dire dans la quasi-totalité des appels.

10.3 Questions ouvertes

1. Une diffusion peut-elle être **annulée** après envoi ? Techniquement oui tant qu'elle n'est pas matérialisée pour tout le monde — mais elle l'aura déjà été pour une partie, et les push seront partis. Un rappel partiel est-il pire que rien ?
2. Faut-il **programmer** une diffusion à une date future ? La file de jobs le permet gratuitement via `run_after`.
3. Combien de temps conserve-t-on les lignes `broadcast_delivery` ? Elles grossissent avec chaque diffusion.
4. Un utilisateur peut-il **se désabonner** des annonces ? Aucune obligation légale pour des notifications de service, mais la question se posera.
5. Plusieurs comptes officiels (support, actualités) — le modèle le permet ; faut-il l'exposer dès maintenant dans l'administration ?

Fin du dossier — étape 3.
Étape suivante : comptes business.